

Cryptographic Settlement Binding for Confidential On-Chain State

Juliano Pereira Sales

Harpo-ZK, Brazil
tzdesing@gmail.com

Marco Túlio Rocha Nascimento

Harpo-ZK, Brazil
marcotulio.nascimento@gmail.com

October 6, 2026

Abstract

A recurring problem in confidential blockchain applications is the need to establish a cryptographic correspondence between a private on-chain obligation created before an external settlement event and an externally observed settlement event that becomes available only later. A zero-knowledge proof can establish the internal consistency of the on-chain state, but it does not by itself establish that an external payment actually occurred. Conversely, an institution may authenticate a statement about an external payment without revealing the payment details on-chain.

We present **cryptographic settlement binding**, a construction that connects these two states while keeping the settlement amount, payment identifier, and counterparty identifier confidential. The construction is two-phase. Before settlement, a commitment c_0 binds the agreed amount and operation identifier to an on-chain obligation. After settlement, a second commitment c binds the same amount and operation identifier to the settlement-side payment identifier. A domain-keyed tag ν provides a deterministic, single-use reference to the settlement identifier without revealing it, while an on-chain domain anchor K_D binds the tag to the registered domain key. A registered payment service provider (PSP) signs an EIP-712 receipt over the commitments rather than over plaintext payment data.

We formalize the resulting properties under Groth16 knowledge soundness, Poseidon and Keccak collision resistance, and ECDSA unforgeability. The construction proves cryptographic consistency and authenticates that a registered PSP signed the accepted statement; it does **not** cryptographically establish that the underlying payment rail was truthful. That remaining correspondence is stated explicitly as an institutional trust assumption. This separation prevents authentication of an institutional statement from being confused with independent evidence of external-world truth.

A reference implementation uses Circom, Groth16 and Solidity and is exercised on a public EVM testnet. The settlement circuit contains 1,251 R1CS constraints at the reported optimization level, and the reference **finalize** transaction costs 353,062 gas. A concrete Pix instantiation is provided as an application profile; the construction itself is independent of Pix.

Keywords: zero-knowledge proofs, confidential transactions, settlement binding, payment attestation, external-state binding, EVM, Groth16, Poseidon, instant payments

1 Introduction

A confidential state transition on a public blockchain can establish that an on-chain operation is internally consistent without revealing the private data on which that consistency depends. It can, for example, prove knowledge of an amount, ownership relation, or transaction witness while exposing only commitments and zero-knowledge proofs. This establishes a property of the blockchain state itself.

A different problem arises when the state transition depends on an event that occurs outside the blockchain. Consider an operation in which an asset or obligation is committed on-chain before an external payment is made. Afterward, an authorized institution observes a settlement event and obtains information identifying that event. The system must then establish that the observed settlement corresponds to the particular confidential obli-

gation that was committed earlier. The difficulty is not merely to prove that the external event exists, nor merely to prove that a private on-chain transition is well formed. It is to establish a cryptographic correspondence between the two events while keeping the sensitive attributes of both confidential.

We call this problem **cryptographic settlement binding**.

Informally, a settlement-binding protocol allows a verifier to establish that an externally observed settlement event is consistent with a confidential state transition previously committed on-chain, while revealing only the information required by the application. The temporal order is essential. Before settlement, the system can commit to the terms of an obligation, but the identifier of the eventual settlement event is not yet available as evidence of that event. After settlement, an authorized observer

can identify and authenticate the external event, but the system must still demonstrate that it corresponds to the state committed earlier.

A straightforward design could reveal the amount and external payment identifier to the blockchain and have a contract compare them with the original obligation. Such a design would establish the desired correspondence, but would defeat the confidentiality requirement. Alternatively, a zero-knowledge proof could hide the values while proving that a private payment identifier is related to an on-chain commitment. This alone, however, does not identify who is authorized to make the external settlement statement, nor does it distinguish cryptographic consistency from the truthfulness of that statement.

Our construction separates these concerns.

The first component is a **two-phase commitment**. Before settlement, an operation commits to its agreed terms as

$$c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}).$$

The commitment is registered on-chain before the external settlement is finalized and can therefore serve as an anchor for an asset escrow or another dependent state transition. After settlement, a second commitment is formed as

$$c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}),$$

where `e2eld` denotes the settlement-side identifier supplied by the concrete payment rail. The settlement proof demonstrates knowledge of a common amount and `txid` underlying both commitments. Thus the external settlement identifier is introduced only in the second phase, while the relationship to the previously committed obligation is established in zero knowledge.

The second component is a **domain-keyed settlement tag**,

$$\nu = \text{Poseidon}(\text{e2eld}, k_D), \quad K_D = \text{Poseidon}(k_D).$$

The tag gives the on-chain protocol a deterministic identifier for the settlement under a particular domain without exposing the underlying settlement identifier. In the reference construction, the tag is consumed at most once by a deployed settlement instance. This provides a cryptographically checkable single-use property while preserving confidentiality of the external identifier. The domain key also makes the linkage selective: an observer that does not possess k_D cannot simply recompute tags for arbitrary settlement identifiers. The resulting property is not global anonymity or unlinkability; those properties are outside the scope of the construction.

The third component is an explicit **institutional trust boundary**. A registered settlement observer signs an authenticated receipt containing the pre-settlement commitment and the public settlement-binding values. In the reference construction, the receipt is represented using EIP-712 and includes an expiration condition. The

zero-knowledge proof establishes consistency between the commitments and the settlement identifier; the signature establishes that the statement was issued by a registered attester. These are cryptographic statements about consistency and authentication.

They are not, by themselves, statements that the external payment rail was truthful.

More precisely, acceptance establishes that the accepted proof is consistent with the pre-settlement commitment and that the corresponding receipt was authenticated by a registered attester. The construction therefore makes the remaining institutional assumption explicit: the authorized attester is assumed to report settlement correctly. This assumption is not hidden inside the zero-knowledge relation and is not presented as a consequence of cryptography. Removing or weakening it is treated separately as an open problem.

This separation is important because an authenticated statement and a truthful statement are different security objects. A cryptographic protocol can prevent an adversary from forging a statement under an honest attester's key; it cannot, without an additional source of trusted evidence, determine whether the institution itself reported a false settlement. The construction consequently reduces the cryptographic problem to authentication and consistency while exposing the institutional assumption as an explicit boundary of the security model.

The paper makes three main contributions.

First, we define and construct cryptographic settlement binding. The construction introduces a two-phase commitment relation in which a confidential obligation is committed before settlement and a subsequent zero-knowledge proof binds the resulting external settlement identifier to the same confidential terms. The construction is expressed independently of a particular payment rail.

Second, we introduce a domain-keyed settlement tag for confidential single-use identification. The tag deterministically represents the external settlement identifier under a domain secret, while its public representation does not disclose that identifier. The security analysis characterizes its binding and single-use properties under the stated cryptographic assumptions and deployment scope.

Third, we make the institutional trust boundary explicit and characterize what the cryptographic protocol does and does not establish. In particular, the security analysis distinguishes authentication of a settlement statement from the truth of the underlying external event. This separation prevents the protocol's cryptographic guarantees from being conflated with assumptions about the behavior of the institution providing the attestation.

The remainder of the paper formalizes these claims. Section 2 defines the settlement-binding model, its participants, state, and security goal. Section 3 gives the construction and its protocol algorithms. Section 4 ana-

lyzes commitment binding, two-phase consistency, attestation authenticity, and single-use settlement. Section 5 characterizes the resulting confidentiality properties and their limits. Section 6 instantiates the construction for Pix, while Section 7 describes policy predicates that can consume the resulting confidential commitments. Section 8 evaluates the reference implementation, including circuit size, gas consumption, and well-determinedness analysis. Section 9 positions the construction relative to external-payment proofs and confidential EVM systems, and Section 10 states the remaining limitations and open problems.

The construction is intentionally narrower than a complete payment system. It does not establish central-bank or payment-rail truth without an additional trusted source, does not provide transaction unlinkability, and does not claim economic finality beyond the state-transition policy of the deployment. Pix is used only as a concrete instantiation of the generic construction in Section 6; the cryptographic construction and its security properties do not depend on Pix-specific semantics.

2 Settlement-Binding Model

2.1 Problem Definition

We consider an application in which a confidential on-chain operation depends on an external settlement event.

Let an operation O have agreed private terms $T = (\text{amount}, \text{txid})$, where amount is the agreed value and txid is an application-level operation identifier. Before the external settlement occurs, the application must be able to create an on-chain state that commits to these terms.

Later, an external payment or settlement rail produces an identifier e2eld , observable by an authorized institution. The identifier e2eld is not itself assumed to prove that settlement occurred: an identifier may exist for an operation that is subsequently rejected or otherwise fails to settle. The external event therefore requires an authenticated statement from an institution that is authorized to observe the rail.

The problem addressed by this paper is to establish a binding between the two states: (1) a confidential commitment to the agreed terms created before settlement; and (2) a confidential commitment incorporating the settlement-side identifier created after settlement.

We call this correspondence **cryptographic settlement binding**.

The construction is intentionally independent of a particular payment rail. A rail-specific deployment supplies an identifier e2eld , a canonical representation of the relevant external fields, and an authorized attester capable of evaluating the required commitment. The core construction does not require the settlement identifier itself to become public.

2.2 Participants and Trust Boundary

The model contains four logical roles.

Obligation issuer. Creates the pre-settlement operation and its commitment. The issuer may be a smart contract, an application, or an escrow component.

Prover. Holds the private witness required to demonstrate that the pre- and post-settlement commitments refer to the same operation.

Attestor. An authorized institution that observes the external settlement system and signs a statement about the observed event.

Verifier. The on-chain contract that accepts or rejects the settlement transition.

The construction separates two claims that are often conflated.

First, the verifier can establish a cryptographic statement:

$$\text{same confidential witness} \Rightarrow \text{consistent pre- and post-settlement commitments.}$$

Second, the verifier can establish an authentication statement:

$$\text{registered attester} \Rightarrow \text{the accepted receipt was signed by an authorized key.}$$

Neither statement alone establishes that the external settlement system actually behaved as claimed. That final correspondence is an institutional assumption about the attester.

Accordingly, the principal trust boundary is:

$$\text{cryptographic consistency} + \text{attester authentication} \neq \text{independent rail truth}$$

The distinction is fundamental to the security claims of the construction.

2.3 Commitments and Domain Separation

Let H be a collision-resistant hash function suitable for use inside the selected zero-knowledge proof system.

For an operation with amount , operation identifier txid , settlement identifier e2eld , and fresh randomness $\text{salt}_0, \text{salt}$, define conceptually:

$$\begin{aligned} c_0 &= H(\text{amount}, \text{salt}_0, \text{txid}) \\ c &= H(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}). \end{aligned}$$

The first commitment represents the obligation before settlement. The second represents the corresponding state after settlement.

A domain D is associated with a secret key k_D and a public anchor

$$K_D = H(k_D).$$

The settlement-side identifier is represented by a domain-keyed tag

$$\nu = H(\text{e2eld}, k_D).$$

The tag provides a public handle that does not reveal $e2eld$ to an observer that does not possess the domain key.

A counterparty commitment may similarly be represented as

$$pc = H(\text{party}, \text{salt}_p),$$

where party is the relevant counterparty identifier and salt_p is fresh randomness.

These equations define the abstract relation that the construction must prove. Their concrete circuit realization is given in Section 3.

2.4 Settlement-Bound Acceptance

An operation is **cryptographically settlement-bound** when an accepting execution establishes all of the following:

1. a valid pre-settlement commitment c_0 was registered for the operation;
2. a valid zero-knowledge witness connects c_0 and c through the same amount and operation identifier;
3. the settlement tag ν is derived from the settlement-side identifier $e2eld$ under the registered domain key;
4. the accepted counterparty commitment is part of the same witness;
5. a registered attester authenticated the same (c_0, ν, pc) tuple accepted by the verifier; and
6. the tag cannot be consumed twice within the deployed settlement instance.

The resulting statement is therefore stronger than either a standalone zero-knowledge transfer proof or a standalone payment signature: it binds an external settlement observation to a previously registered confidential on-chain obligation.

2.5 Security Properties

The formal analysis in Section 4 considers four principal properties.

Commitment consistency. An accepting proof cannot substitute different values or operation identifiers between the pre- and post-settlement commitments, except with negligible probability under the stated cryptographic assumptions.

Settlement-identifier binding. An accepting proof binds the public settlement tag to the settlement identifier contained in its witness under the registered domain key.

Attestation binding. An accepting settlement is associated with a receipt signed by a registered attester over the same public commitments accepted by the verifier.

Instance-scoped single use. A settlement tag accepted by one deployed settlement instance cannot be consumed again by that instance.

The properties deliberately do not include a claim that the external payment rail itself is truthful. That statement belongs to the institutional assumption described next.

2.6 Institutional Assumption

Let I_1 denote the trusted-attester assumption.

A registered attester signs $settled = true$ only for an external payment that it observed as settled and credited, and maintains an authoritative internal mapping between the operation identifier, settlement identifier, amount, and relevant counterparty information.

Under I_1 , the attestation provides an authenticated bridge from the external event to the cryptographic construction.

Without I_1 , the protocol still authenticates that a registered key signed the statement, but it cannot distinguish a truthful statement from a deliberately false one produced by a malicious or compromised attester.

This distinction is preserved throughout the remainder of the paper.

2.7 What the Model Does Not Claim

The model does not assume that the external rail exposes a cryptographically verifiable proof to the public EVM. It also does not provide economic finality, anonymity, or unlinkability merely from the use of zero-knowledge proofs.

In particular, confidentiality of amount, $e2eld$, and party from ordinary chain observers is distinct from unlinkability between operations. Holders of the domain key can intentionally use ν as a selective-linkability mechanism.

The concrete construction implementing this model is specified in Section 3.

3 Settlement-Binding Construction

This section instantiates the abstract model of Section 2 as a concrete EVM-compatible construction using Poseidon commitments [2], a Groth16 proof [1], a domain-keyed settlement tag, and an EIP-712 attestation receipt. The construction is represented by the tuple

$$\Pi = (\text{Setup}, \text{Commit}, \text{Prove}, \text{Verify}, \text{Attest}, \text{Settle}).$$

The algorithms are not intended to introduce a new zero-knowledge proof system. They specify how standard commitments, a Groth16 proof, an institutional attestation, and an on-chain state transition are composed to realize the settlement-binding relation defined in Section 2.

3.1 Setup

Setup is the composition of two independent sub-algorithms,

$$\text{Setup} \rightarrow (\text{Setup_ZK}, \text{Setup_Domain}),$$

which are not run together and do not share randomness or inputs.

Setup_ZK.

$$\text{Setup_ZK}(1^\lambda) \rightarrow (pp, vk)$$

generates the Groth16 proving and verification parameters for the settlement circuit. This step requires a trusted-setup ceremony and is run once per circuit, independently of any settlement domain. The same (pp, vk) pair is reused by every domain that deploys the same circuit.

Setup_Domain.

$$\text{Setup_Domain}(1^\lambda, D) \rightarrow (k_D, K_D)$$

samples a fresh domain secret k_D for a settlement domain D and computes the public domain anchor

$$K_D = \text{Poseidon}(k_D).$$

Creating a domain never requires a new Groth16 ceremony. The deployed verifier stores K_D as the domain anchor; rotating k_D creates a new domain rather than mutating the existing anchor. A deployment may create many domains against a single (pp, vk) produced once by Setup_ZK.

3.2 Commit

For agreed private terms $T = (\text{amount}, \text{txid})$, the commitment algorithm samples fresh salt_0 and computes

$$\begin{aligned} c_0 &= \text{Commit}_0(\text{amount}, \text{txid}; \text{salt}_0) \\ &= \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}). \end{aligned}$$

The application registers c_0 with the on-chain settlement instance and locks the corresponding operation. At this point the settlement-side identifier e2eld is not required by the on-chain state: the operation establishes its cryptographic identity before the external event occurs.

After settlement, the prover samples fresh salt and computes

$$\begin{aligned} c &= \text{Commit}_1(\text{amount}, \text{e2eld}, \text{txid}; \text{salt}) \\ &= \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}), \end{aligned}$$

where e2eld is the settlement-side identifier obtained from the external payment rail.

The two commitments are intentionally distinct: c_0 identifies the obligation before settlement, while c incorporates the external settlement identifier. The same

private amount and txid are reused across both, rather than independently supplied, which is what gives the construction its binding property (made explicit in Section 3.7).

3.3 Prove

The prover constructs the witness

$$w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k_D, \text{party}, \text{salt}_p)$$

and computes

$$\begin{aligned} \nu &= \text{Poseidon}(\text{e2eld}, k_D), \\ pc &= \text{Poseidon}(\text{party}, \text{salt}_p). \end{aligned}$$

The public statement is

$$y = (c_0, c, \nu, K_D, pc).$$

The prover then computes

$$\pi \leftarrow \text{Prove}(pp, w; y).$$

The circuit accepts the witness only if

$$c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}), \quad (1)$$

$$c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}), \quad (2)$$

$$\nu = \text{Poseidon}(\text{e2eld}, k_D), \quad (3)$$

$$K_D = \text{Poseidon}(k_D), \quad (4)$$

$$pc = \text{Poseidon}(\text{party}, \text{salt}_p). \quad (5)$$

The same private amount and txid are structurally constrained in both phases: a valid proof does not merely show that two independently supplied hashes are valid, but that both commitments were derived from the same private amount and operation identifier.

The settlement tag ν is intentionally scoped to the domain key k_D rather than computed directly from e2eld . This prevents an observer who already knows a candidate e2eld from testing it directly against on-chain tags without knowledge of k_D , and it makes the single-use property enforced on ν (Section 3.6) instance-scoped rather than globally unique across independently deployed domains.

The counterparty commitment pc is likewise not published in plaintext; it is cross-checked against the PSP's own commitment at settlement time (Section 3.5), which is what makes downstream predicates over pc meaningful rather than decorative — without that cross-check, a prover could generate a valid settlement proof while associating it with an unrelated counterparty commitment.

3.4 Verify

The proof verifier evaluates

$$\text{Verify}(vk, y, \pi) \in \{0, 1\}.$$

Acceptance establishes only the cryptographic relation represented by the circuit. In particular, successful verification does not independently establish that the external settlement identified by e2eld actually occurred. The latter statement enters through *Attest*.

3.5 Attest

A registered payment institution evaluates the external event and constructs the typed receipt

$$R = (c_0, \nu, pc, settled, deadline).$$

The attester signs the receipt using EIP-712:

$$\sigma \leftarrow \text{Sign}_{sk_P}(R).$$

The attestation therefore authenticates the public commitments rather than exposing the amount, settlement identifier, or counterparty identifier.

The attester is assumed to derive c_0 from the amount it actually observed for the external settlement — independently recomputing

$$c'_0 = \text{Poseidon}(\text{amount_settled}, \text{salt}_0, \text{txid})$$

and signing only when c'_0 matches the operation's registered c_0 . This gives the construction its **commitment-carried value-consistency** property: the attester authenticates a commitment derived from the observed amount without the amount itself ever appearing in the receipt.

The receipt also carries an expiration deadline; a stale receipt cannot be used to finalize an operation after its authorization period.

3.6 Settle

The on-chain settlement algorithm evaluates

$$\text{Settle}(c_0, \nu, pc, \pi, R, \sigma).$$

It accepts only if:

1. the operation is currently **Locked**;
2. σ authenticates a currently registered attester;
3. $settled = \text{true}$;
4. the receipt has not expired;
5. the receipt's c_0 equals the locked commitment;
6. the receipt's pc equals the proof's public pc ;
7. the proof's K_D equals the registered domain anchor;
8. $\text{Verify}(vk, y, \pi) = 1$; and
9. ν has not already been consumed.

Upon acceptance, the contract records the settlement result and consumes ν , transitioning the operation to **Finalized**.

The ordering of the checks is also relevant operationally: inexpensive receipt and state checks (1–7) can reject malformed submissions before the pairing-based proof verification (8) is executed.

3.7 Construction Invariant

The central invariant of the construction is that

$$\begin{aligned} c_0 &= \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}) \\ c &= \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}) \end{aligned}$$

share the private values `amount` and `txid`. Because the same private amount and txid occur in both relations, an accepting proof establishes that the pre-settlement and post-settlement commitments refer to the same agreed value and operation identifier, subject to the cryptographic assumptions stated in Section 4.

The additional relations

$$\begin{aligned} \nu &= \text{Poseidon}(\text{e2eld}, k_D) \\ K_D &= \text{Poseidon}(k_D) \end{aligned}$$

bind the public settlement tag to the registered domain.

Finally, the attestation relation binds the accepted public tuple (c_0, ν, pc) to a registered institutional key.

The security consequences of these relations are proved in Section 4.

3.8 Reversal and Liveness

A valid attestation does not necessarily imply economic irreversibility of the external settlement.

The reference deployment therefore distinguishes **Finalized** from **Irrevocable**.

Within the configured dispute window, the authorized attester may register a reversal. A reversed operation cannot become releasable merely because the nominal window subsequently expires.

Separately, an operation that remains locked without successful settlement can expire through the protocol timeout. This prevents an unavailable or non-cooperating attester from permanently locking the pre-settlement obligation.

The timeout is a liveness mechanism; it is not evidence about the external payment rail.

3.9 Protocol Summary

The resulting sequence realizes Π end to end:

1. run **Setup_Domain** to register the domain anchor K_D (**Setup_ZK** having already produced (pp, vk) once for the circuit);
2. run **Commit** to compute c_0 from the agreed amount and operation identifier, and lock the operation against c_0 ;
3. observe the external settlement and obtain the settlement-side identifier **e2eld**;
4. run **Commit** to construct c , and run **Prove** to construct ν , pc , and the proof π ;

5. obtain the PSP receipt via `Attest`, over $(c_0, \nu, pc, settled, deadline)$;
6. submit the receipt and proof to `Settle`;
7. `Settle` verifies the attestation, public commitments, domain anchor and proof, consumes ν , and finalizes the operation.

The construction establishes the cryptographic relation between the pre-settlement obligation and the post-settlement state. Section 4 analyzes the security properties of this relation.

4 Security Analysis

This section characterizes the security guarantees provided by the settlement-binding construction. The analysis separates three different claims that are easily conflated:

- commitment binding, which prevents a commitment from being opened to different values;
- two-phase consistency, which binds the post-settlement state to the pre-settlement obligation; and
- authenticated settlement binding, which connects the resulting confidential state to an attestation produced by an authorized PSP.

We additionally analyze single-use settlement tags and state the main settlement-binding theorem.

The analysis uses the following assumptions.

- **(A1) Groth16 knowledge soundness.** The deployed Groth16 system is knowledge-sound for the settlement circuit under its trusted setup.
- **(A2) Poseidon collision resistance and random-oracle privacy heuristic.** It is computationally infeasible to find distinct inputs producing the same Poseidon output. For the confidentiality discussion only, Poseidon is modeled as a random oracle, with fresh high-entropy salts used as blinding values.
- **(A3) ECDSA/EIP-712 unforgeability.** Signatures produced under the registered PSP keys satisfy EUF-CMA security.
- **(A4) Charge-identifier uniqueness.** Within a domain, a txid identifies a single payment obligation and is not legitimately reused for distinct charges.
- **(A5) Keccak-256 collision resistance.** The hash used by the attestation oracle to bind the off-chain receipt data is collision resistant.

We also distinguish these cryptographic and operational assumptions from the institutional assumption that a registered PSP reports settlement correctly. The latter cannot be discharged by the cryptographic construction.

4.1 Commitment binding

The first property concerns the individual commitments independently of the settlement protocol.

Property 1 (Commitment binding). *No probabilistic polynomial-time adversary can produce two distinct openings of the same c_0 or c , except with negligible probability.*

For c_0 , suppose an adversary produces $(\text{amount}_1, \text{salt}_{0,1}, \text{txid}_1)$ and $(\text{amount}_2, \text{salt}_{0,2}, \text{txid}_2)$ such that $\text{Poseidon}(\text{amount}_1, \text{salt}_{0,1}, \text{txid}_1) = \text{Poseidon}(\text{amount}_2, \text{salt}_{0,2}, \text{txid}_2)$ while the two tuples differ. This constitutes a collision in Poseidon.

The same argument applies to c .

Therefore,

$$\Pr[\text{Bad}_{CB}] \leq \text{Adv}_{\text{Poseidon}}^{CR}(A). \quad (6)$$

This property is independent of Groth16, the PSP attestation, and the settlement contract. It is a property of the commitment function itself.

4.2 Two-phase consistency

Commitment binding alone is insufficient. The central property of the construction is that the same confidential obligation is represented before and after settlement.

Recall that $c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid})$ and $c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid})$.

The circuit contains a single witness variable `amount` and a single witness variable `txid`, which are used in both equations.

Theorem 1 (Two-phase consistency). *Assume Groth16 knowledge soundness and Poseidon collision resistance. Except with negligible probability, every accepting proof contains a witness for which the amount and txid used in c are identical to those used in c_0 .*

Consequently, if an escrow was created against $c_0 = \text{Poseidon}(\text{amount}^*, \text{salt}_0^*, \text{txid}^*)$, an accepting proof cannot use an amount or txid different from $(\text{amount}^*, \text{txid}^*)$ in the post-settlement commitment, except with negligible probability.

Proof. Consider an accepting proof π . By Groth16 knowledge soundness, except with negligible probability an extractor obtains a witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k_D, \text{party}, \text{salt}_p)$ satisfying the circuit relation.

Because the circuit contains one witness variable `amount`, that same value occurs in both commitment equations. Likewise, a single witness variable `txid` occurs in both equations. There are therefore no independent values $\text{amount}_0, \text{amount}_1$ or $\text{txid}_0, \text{txid}_1$ that the prover can select independently.

It remains to relate the extracted witness to the escrowed commitment. If $(\text{amount}, \text{txid}) \neq (\text{amount}^*, \text{txid}^*)$, then the extracted witness and the

escrow-opening values define two distinct preimages of the same c_0 . This contradicts Poseidon collision resistance.

Hence

$$\Pr[\text{Bad}_{2P}(A)] \leq \text{Adv}_{\text{Groth16}}^{KS}(A) + \text{Adv}_{\text{Poseidon}}^{CR}(A). \quad (7)$$

□

The implementation audit confirms that this equality is enforced structurally by the circuit: amount and txid are the same private witness variables in both commitment computations rather than values compared by an external software check.

This property is deliberately weaker than a statement about real-world settlement. It establishes consistency between the two confidential commitments; whether the corresponding external event actually settled is handled separately by the attestation layer.

4.3 Authenticated settlement binding

The previous theorem establishes consistency between the two phases but does not authenticate the external settlement observation.

The attestation layer adds this missing link.

Property 2 (Authenticated settlement binding). *An accepted operation is settlement-bound if there exist: a registered PSP P ; an unexpired receipt $R = (c_0, \nu, pc, \text{true}, \text{deadline})$; a valid PSP signature $\sigma = \text{Sign}_P(R)$; and a Groth16 witness satisfying the settlement circuit, such that the receipt and the accepted proof refer to the same (c_0, ν, pc) .*

This property intentionally does not state that the PSP’s assertion *settled* = *true* is objectively true. That fact belongs to the institutional trust boundary.

Theorem 2 (Settlement binding). *Condition on deterministic correctness of the deployed settlement contract and immutability of the registered domain anchor K_D . Under assumptions (A1), (A2), (A3), and (A5), for every probabilistic polynomial-time adversary A ,*

$$\begin{aligned} \Pr[\text{Bad}_{SB}(A)] \leq & \text{Adv}_{\text{ECDSA}}^{\text{EUF-CMA}}(A) \\ & + \text{Adv}_{\text{Groth16}}^{KS}(A) + \text{Adv}_{\text{Poseidon}}^{CR}(A) \\ & + \text{Adv}_{\text{Keccak}}^{CR}(A). \end{aligned} \quad (8)$$

Here Bad_{SB} is the event that Settle accepts an operation for which either: no registered PSP signed the accepted tuple $(c_0, \nu, pc, \text{true}, \text{deadline})$, or the accepted tag ν is not the value $\text{Poseidon}(\text{e2eld}, k_D)$ for the e2eld contained in an accepting witness under the registered domain key.

Proof sketch. Suppose Settle accepts. First, the deployed contract verifies the Groth16 proof against (c_0, c, ν, K_D, pc) . By knowledge soundness, except with negligible probability, an extractor obtains a witness

satisfying the circuit equations. In particular, $\nu = \text{Poseidon}(\text{e2eld}, k_D)$ and $K_D = \text{Poseidon}(k_D)$. If the witness used a domain key different from the registered key while producing the same K_D , this would yield a Poseidon collision except with negligible probability.

Second, the contract requires a valid attestation for the same c_0, ν , and pc . If an adversary produces an accepted receipt without a registered PSP having signed it, the adversary has forged an EIP-712/ECDSA signature, contradicting (A3).

Third, the attestation oracle binds its off-chain receipt information to the corresponding tag using its binding hash. A successful substitution that preserves the accepted digest while changing the bound values would contradict the collision resistance of Keccak-256.

Therefore, except with negligible probability, an accepted operation has both a valid ZK witness and a valid registered-PSP attestation for the same public tuple. □

The theorem establishes cryptographic consistency and attestation authenticity. It does not establish that the PSP itself is honest.

4.4 Single-use settlement

The settlement tag provides replay protection.

For a fixed domain key, $\nu = \text{Poseidon}(\text{e2eld}, k_D)$ is deterministic in the settlement identifier. Consequently, the same e2eld produces the same ν within that domain.

The public anchor K_D prevents a prover from selecting a different domain key while still satisfying the same public statement, except with negligible probability under Poseidon collision resistance.

The reference implementation enforces single use at two independent points: the settlement contract records accepted tags as consumed (**tagUsed**), and the attestation oracle separately refuses to attest a tag that it has already attested, regardless of which commitment the new attestation is for. In the deployment topology used throughout this paper — one oracle per deployed settlement instance — the oracle’s check is the one that actually fires: a second settlement for the same tag cannot obtain a valid attestation at all, so it never reaches the tag-consumption check in the settlement contract. The tag-consumption check in the settlement contract is retained as defense in depth and becomes load-bearing only in a deployment where several settlement contracts share a single oracle.

Theorem 3 (Instance-scoped settlement uniqueness). *Fix a domain anchor K_D and a deployed (oracle, Settle) instance pair. Under domain-key immutability, Poseidon collision resistance, single-use enforcement of ν , and the charge-identifier uniqueness assumption, no probabilistic polynomial-time adversary can cause two accepted settlements corresponding to the same e2eld , except with negligible probability.*

Proof sketch. Suppose two settlements correspond to the same $e2eld$. Because the domain key is fixed, $Poseidon(e2eld, k_D)$ produces the same ν for both. The public anchor K_D prevents the two accepting proofs from using unrelated domain keys except with negligible probability. The first accepted settlement is attested and consumes the tag at the oracle. The second settlement, for a different commitment but the same tag, cannot obtain an attestation from that oracle; absent an attestation it cannot satisfy the finalize precondition that requires one. Hence two accepted settlements for the same $e2eld$ cannot occur within the same deployed (oracle, **Settle**) instance pair, except with negligible probability. $\square$

This was confirmed experimentally in the reference implementation: a second settlement attempt over a different commitment carrying the same tag is rejected by the oracle’s attestation call itself, before the settlement contract’s own finalize check is reached.

The scope qualifier is important. The reference implementation does not provide a globally shared spent-tag registry, at either the oracle or the settlement-contract level. Consequently, the property does not extend automatically across two independently deployed (oracle, **Settle**) instance pairs, nor across domains with different keys. This was also confirmed experimentally: two independently deployed (oracle, **Settle**) pairs, each given the same commitment, tag, and proof, both accept the settlement, because neither oracle has prior knowledge of the other’s attestation. A shared oracle, or a shared registry indexed by K_D , would be required to elevate this property from instance scope to domain scope.

4.5 Attestation authenticity versus settlement truth

The construction deliberately separates two statements that are often conflated: “a registered PSP signed ‘settled’” and “the external payment rail actually settled.” The first is a cryptographic property. The second is an institutional property.

Property 3 (Attestation authenticity). *If **Settle** accepts an operation, then, except with negligible probability under ECDSA/EIP-712 EUF-CMA security, a registered attester produced the accepted receipt.*

This follows directly from signature verification, registration checking, domain separation, and expiry checking.

However, a malicious or compromised registered PSP can still produce a valid signature for a false settlement claim. No cryptographic reduction in this construction eliminates that possibility.

We therefore model the remaining trust as:

- (I1) Registered PSP honestly reports observed settlement.

Under (I1), the cryptographic settlement-binding theorem can be interpreted operationally as:

$$\begin{aligned} \text{Accepted} &\Rightarrow \text{ConfidentialStateConsistent} \\ &\wedge \text{PSPAuthenticated} \\ &\wedge \text{SettlementObserved}_{PSP}. \end{aligned}$$

Without (I1), only the first two terms are established.

This distinction is essential to the scope of the construction and prevents the protocol from being interpreted as a zero-knowledge proof of central-bank or payment-rail truth. The original analysis explicitly identifies this residual PSP assumption as outside the cryptographic reduction.

4.6 Main settlement-binding theorem

We can now state the principal security claim of the construction.

Theorem 4 (Cryptographic settlement binding). *Assume: Groth16 knowledge soundness for the deployed settlement circuit; Poseidon collision resistance; ECDSA/EIP-712 EUF-CMA security; Keccak-256 collision resistance for the attestation binding hash; uniqueness of the payment-obligation identifier (txid); an immutable domain anchor K_D ; and deterministic correctness of the deployed **Settle** contract. Then every accepted settlement is, except with negligible probability, cryptographically consistent with the pre-settlement commitment and authenticated as the same (c_0, ν, pc) tuple by a registered PSP. More precisely,*

$$\begin{aligned} \Pr[\text{Bad}_{SB}(A)] &\leq \text{Adv}_{\text{ECDSA}}^{\text{EUF-CMA}}(A) \\ &\quad + \text{Adv}_{\text{Groth16}}^{KS}(A) + \text{Adv}_{\text{Poseidon}}^{CR}(A) \\ &\quad + \text{Adv}_{\text{Keccak}}^{CR}(A). \end{aligned} \quad (9)$$

The operational assumption on txid uniqueness is required to interpret the identifier as a unique payment obligation, while the institutional assumption that the PSP truthfully reports settlement remains outside the cryptographic bound.

The theorem therefore establishes the following implication:

$$\begin{aligned} \text{SettleAccept} &\Rightarrow \exists w \text{ such that } c_0 = \text{Commit}_0(w), \\ &c = \text{Commit}(w), \nu = \text{Tag}(w), \text{ and } pc = \\ &\text{PartyCommit}(w); \text{ and there exists a registered PSP} \\ &P \text{ such that } P \text{ signed } (c_0, \nu, pc, \text{true}, \text{deadline}). \end{aligned}$$

The theorem does not claim that the payment amount, payment identifier, or counterparty is publicly recoverable from the accepted transcript. Nor does it claim economic irreversibility of the underlying payment. Those are separate properties considered in the privacy and limitations analyses.

The significance of the construction is therefore not that it removes the PSP from the trust model. Rather, it

converts an external settlement observation into a cryptographically constrained statement about a confidential on-chain state transition, while preserving the confidentiality of the underlying settlement data.

5 Privacy Analysis

The settlement-binding construction is designed to hide the sensitive attributes of the external payment while preserving publicly verifiable evidence that the settlement is bound to the confidential state transition.

The privacy claim is deliberately narrower than anonymity or unlinkability.

5.1 Confidential values

The following values are private witness elements: amount, e2eld , txid , party. They are never supplied as public inputs to the settlement verifier.

The public transcript instead contains c_0, c, ν, K_D, pc , together with the zero-knowledge proof and the PSP’s signed receipt.

The amount appears only through the commitments c_0 and c , where the salts provide high-entropy blinding.

Likewise, the counterparty appears only through $pc = \text{Poseidon}(\text{party}, \text{salt}_p)$, and the payment identifier appears only through the keyed tag $\nu = \text{Poseidon}(\text{e2eld}, k_D)$.

For this confidentiality discussion, we use a random-oracle heuristic in which the fresh high-entropy salts act as blinding values. Together with the zero-knowledge property of Groth16, this prevents the public transcript from directly revealing the corresponding witness values. This is a privacy heuristic rather than a standalone composable hiding theorem for Poseidon.

The PSP receipt does not contain the amount, e2eld , txid , or plaintext counterparty identifier. It therefore does not introduce a plaintext disclosure channel.

5.2 What the construction does not hide

The construction does not provide complete transaction anonymity. In particular, the following remain public or potentially observable: existence of the on-chain operation; timing of the settlement transaction; submitting and relaying addresses; the registered attester address; the domain anchor K_D ; repeated counterparty commitments (pc); ordinary transaction-level metadata.

Therefore, confidentiality $\neq$ unlinkability.

An observer may be unable to recover the amount or payment identifier while still correlating multiple operations through public metadata.

This is intentional. The construction provides confidential settlement binding, not an anonymity system.

5.3 Selective linkability of the settlement tag

The keyed tag $\nu = \text{Poseidon}(\text{e2eld}, k_D)$ provides a more specific privacy property.

An observer outside the domain who does not know k_D cannot directly compute candidate tags for known e2eld ’s. In contrast, an institution holding k_D can compute $\text{Poseidon}(\text{e2eld}, k_D)$ for e2eld values it legitimately observes and therefore correlate those payments with on-chain tags.

The tag should therefore be understood as a selective-linkability mechanism, rather than as a universal privacy mechanism.

This distinction is important because the PSP and proving institutions are already trusted participants in the payment domain and necessarily possess information that an external blockchain observer does not.

5.4 Counterparty commitments

The counterparty commitment provides a reusable pseudonymous anchor for the settlement participant: $pc = \text{Poseidon}(\text{party}, \text{salt}_p)$.

Its purpose is not to prove the legal identity of a person or organization on-chain. Rather, it provides a confidential reference against which zero-knowledge policy predicates may subsequently be evaluated.

The commitment is meaningful only because the PSP computes it from the debtor identifier contained in the payment message and the finalization procedure requires the attested pc to equal the public pc of the settlement proof.

Without this cross-check, a prover could select a favorable counterparty independently of the actual payer.

5.5 Privacy boundary

The construction consequently claims confidentiality of settlement attributes, not anonymity of the settlement transaction itself.

6 Pix Instantiation

The general settlement-binding construction is instantiated using Pix as the external settlement rail.

The abstraction requires three rail-side capabilities: a payment-obligation identifier available before settlement; a settlement-specific identifier available to an authorized observer of the payment; and an attester capable of confirming the settlement and recomputing the relevant commitment.

For Pix [14], the payment-obligation identifier is the merchant-side txid , while the settlement-specific identifier is the EndToEndId (e2eld). A regulated PSP acts as the attester.

Property	Status
Amount confidentiality	Protected
e2eld confidentiality	Protected
txid confidentiality	Protected
Counterparty identifier confidentiality	Protected
Operation existence	Not protected
Transaction timing	Not protected
Submitter/relayer privacy	Not protected
PSP identity privacy	Not protected
Cross-operation unlinkability	Not guaranteed

Table 1: Privacy boundary of the settlement-binding construction.

The distinction between the two identifiers is important.

The txid belongs to the charge and can therefore participate in the pre-settlement commitment: $c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid})$.

The e2eld is associated with the actual payment transaction and therefore enters the post-settlement commitment: $c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid})$.

The construction does not rely on the identifier itself being evidence of settlement. An e2eld can exist for a payment that is subsequently rejected. Settlement status is therefore supplied by the PSP attestation rather than inferred from possession of the identifier.

This makes the Pix instantiation representative of a broader class of instant-payment systems. The same architecture could apply to another rail if it exposes a unique settlement identifier, a pre-settlement obligation identifier, and an authorized institution capable of attesting the resulting settlement.

6.1 Settlement semantics

The reference implementation defines *settled* = *true* to mean that the receiver has credit availability, rather than merely that the payment has cleared an intermediate rail state.

This distinction matters because a payment can remain subject to return or fraud-recovery mechanisms after credit becomes available.

The protocol therefore distinguishes: **Finalized** (the cryptographic settlement-binding conditions were satisfied) and **Irrevocable** (the protocol’s configured dispute window has elapsed without an accepted reversal).

Irrevocable is a protocol state and must not be interpreted as a claim of economic irreversibility by the payment rail.

The reference deployment uses a 72-hour dispute window. The underlying rail may permit certain return or fraud-recovery mechanisms for substantially longer periods. Consequently, an application requiring economic finality must choose its own risk policy rather than treating the protocol state as a substitute for the rail’s legal or operational finality.

7 Policy Predicates as an Application Layer

The settlement-binding primitive can be consumed by policy predicates without exposing the underlying settlement data.

These predicates are intentionally treated as an application layer, not as part of the core settlement-binding theorem.

Three reference predicates are considered.

7.1 Value bound

A range circuit proves that the committed settlement amount is below a policy limit: $\text{amount} < \text{limit}$.

The predicate uses a bounded comparison over the opening of the settlement commitment.

The statement is deliberately called a policy predicate rather than a universal compliance determination. It establishes only that the amount satisfies the specified limit under the committed value.

7.2 Sanctions-set non-membership

A Sparse Merkle Tree represents an authority-defined set of sanctioned identifiers.

The prover demonstrates that the committed counterparty does not occur in the tree: $\text{party} \notin S$.

The public statement consists of the counterparty commitment and the policy root.

This does not establish that the counterparty is legally “unsanctioned” in an absolute sense. The result is relative to the particular root, its version, its normalization procedure, and the authority responsible for maintaining it.

7.3 KYC-set membership

The dual predicate proves $\text{party} \in \text{KYCRoot}$.

Again, the cryptographic statement is set membership. It does not independently establish the correctness, freshness, jurisdictional scope, or lifecycle of the underlying KYC process.

7.4 Binding predicates to the actual payer

The important architectural point is that the predicates operate on the settlement’s attested pc , rather than on an arbitrary prover-selected identity.

The settlement protocol first establishes $pc = \text{Poseidon}(\text{party}, \text{salt}_p)$ and requires the PSP attestation to contain the same pc .

A consumer of a policy predicate must then verify that its public commitment corresponds to the pc of the settled operation.

Thus the composition is: $\text{PSP-attested payer} \rightarrow pc \rightarrow \text{ZK policy predicate}$.

Without this composition check, a prover could demonstrate a predicate about a different identity.

The predicates are therefore useful as an extension of the settlement-binding primitive, but they do not enlarge the cryptographic claim of the core construction.

8 Implementation and Evaluation

8.1 Reference implementation

The settlement circuit is implemented in Circom 2.2.1, the contracts use Solidity 0.8.27, and the Groth16 verifier is generated with snarkjs 0.7.5, matching the toolchain versions pinned in the reference repository.

The primary circuit is `settlement_verify_v2`.

At optimization level `-02`, it contains 1,251 R1CS constraints. At `-01`, the circuit contains 1,266 nonlinear and 1,524 linear constraints.

The circuit exposes five public signals and eight private witness values.

The circuit contains five Poseidon gadgets with arities 3, 4, 2, 1, 2, corresponding to the two commitments, settlement tag, domain anchor, and counterparty commitment.

8.2 On-chain cost

The primary figure is the complete settlement transaction rather than an isolated verifier call.

On the XDC Apothem public testnet, chain ID 51, the measured transaction costs were:

These four figures are reconfirmed against the recorded Apothem deployment log and the automated test suite. `cancel(c0)` and `attestReversal`, which appeared in an earlier draft of this table, are omitted here because they were not reconfirmed against the Apothem deployment; a local-network reproduction gives 28,115 and 57,541 gas respectively, close to but not identical to the previously recorded Apothem figures (30,259 and 63,529), and the discrepancy has not been attributed to a specific cause.

The complete `finalize` transaction is the primary deployment metric because it includes proof verification,

attestation cross-checking, domain-anchor validation, tag consumption, and state transition.

The isolated verifier transaction includes its own base transaction cost and proof calldata and therefore cannot simply be subtracted from the complete `finalize` transaction to derive a verifier-only execution cost.

Groth16 pairing verification remains the dominant component of the settlement cost.

8.3 Circuit well-determinedness

The circuits were additionally analyzed with Ecne [11] for well-determinedness.

This analysis is explicitly not presented as formal verification of functional correctness. It addresses the narrower question of whether circuit signals are sufficiently constrained.

The meaningful `-00` results were:

For the optimized settlement circuit, Ecne solved 4/5 declared targets and therefore returned inconclusive, not unsound.

This distinction is important: the optimized result does not constitute a negative security result. It indicates that the symbolic analysis could not determine one target under the optimized constraint representation.

The analysis also exposed a methodological pitfall in which circuits whose public outputs are expressed only as input-plus-equality constraints can produce a vacuous Ecne verdict. The paper therefore reports the number of targets explicitly rather than relying on the tool’s raw “sound” output.

The well-determinedness analysis establishes the absence of a particular class of under-constrained-circuit bugs. It does not establish functional conformance with the intended protocol specification.

8.4 End-to-end deployment

The construction was deployed and exercised on the XDC Apothem public testnet.

The complete flow was executed: `lock c0` $\rightarrow$ PSP receipt $\rightarrow$ attestation $\rightarrow$ Groth16 proof $\rightarrow$ finalize.

No amount, `e2eld`, or charge identifier was included in the corresponding settlement calldata.

Negative on-chain tests exercised: non-attestor signatures, `settled = false`, expired receipts, mismatched `c0`, mismatched counterparty commitment, incorrect domain key, reversal, and premature and post-deadline cancellation. These cases are exercised by a manually run script against the Apothem deployment and are not part of the automated test suite.

Each negative case reverted as intended in the reference deployment.

Tag replay is additionally covered by an automated test against a local deployment of the same contracts, using two independently generated Groth16 proofs that share an `e2eld` and domain key and therefore share a settlement

Operation	Gas
<code>lock(c0, deadline)</code>	74,982
<code>attest(receipt, signature)</code>	90,913
<code>finalize(proof)</code>	353,062
<code>isolated verifyProof</code>	243,182

Table 2: Measured XDC Apothem gas costs.

Circuit	Targets solved	Signals	Result
<code>settlement_verify_v2_out</code>	5/5	4,229/4,229	Sound
<code>kyc_inclusion_verify_out</code>	1/1	18,230/18,248	Sound

Table 3: Ecne -00 well-determinedness results.

tag while committing to different terms. A second settlement for the same tag is rejected, and — as discussed in Section 4.4 — the rejection occurs at the attestation oracle rather than at the settlement contract’s own tag-consumption check, which only becomes load-bearing when several settlement contracts share one oracle. The same automated test also confirms, by constructing two independently deployed (oracle, Settle) pairs, that this single-use property does not extend across independent deployments, consistent with the instance-scoped qualifier in Theorem 3.

The reproducibility claims in this section refer to the repository state identified by commit `0db97bf91ddc5be475e3d4175a62e241eb80df79`.

The prototype deployment uses the same key for deployment, attestation registration, and registry administration. This is explicitly an implementation convenience and not a design requirement. A production deployment should separate these roles and use appropriate governance controls.

9 Related Work

9.1 External payment evidence

The closest prior work is ZKP2P [6], which demonstrates that external payment evidence can be used to authorize an on-chain action.

The present construction does not claim novelty for the general pattern “external payment evidence $\rightarrow$ on-chain action.”

Its target is narrower: pre-settlement confidential commitment $\rightarrow$ external settlement observation $\rightarrow$ confidential on-chain state.

The distinguishing feature is therefore the temporal and cryptographic relationship between the state committed before settlement and the external settlement identifier available afterward.

The attestation model also differs: the reference construction uses a regulated PSP as the institutional ob-

server rather than consumer-facing email or TLS evidence.

9.2 Confidential EVM transfers

Systems such as Zeto [5], Nightfall, Railgun, and Rayls/Enyigma [3, 4] address confidential transfer validity.

Their core question is approximately: “Is this private on-chain state transition internally valid?”

The settlement-binding construction addresses a different question: “Does a specific externally observed settlement correspond to this confidential on-chain state?”

The construction is therefore complementary to confidential asset systems rather than a replacement for them.

In particular, the resulting commitment can serve as an upstream input to a confidential asset protocol.

No direct integration with Zeto or Paladin is claimed in this paper.

9.3 Institutional privacy and delivery-versus-payment

Canton [7] and the Drex ecosystem address institutional privacy and delivery-versus-payment in permissioned or regulated infrastructures.

The reference construction instead places the settlement-binding layer on a public EVM chain while using an institutional PSP as the attester.

The resulting trust boundary is therefore different: public EVM + regulated attestation + zero-knowledge confidentiality.

9.4 Confidential compliance and auditability

Zeto [5] and Qurrency [10] provide related mechanisms for confidential compliance and auditor access.

The policy predicates in this work differ primarily in their disclosure semantics: the consumer can learn a

boolean predicate result without necessarily receiving the underlying amount or identity.

The post-quantum audit channel is similarly complementary rather than central to the settlement-binding contribution.

9.5 Positioning

The scientific contribution should therefore not be stated as a collection of novel cryptographic primitives.

Poseidon commitments [2], Groth16 [1], EIP-712 signatures, nullifier-like tags, Sparse Merkle Trees, and confidential transfer mechanisms are individually established.

The contribution is their composition around a specific missing interface: external settlement event $\rightarrow$ cryptographic settlement binding $\rightarrow$ confidential EVM state.

This positioning is also why the Pix implementation is an instantiation of the construction rather than the construction itself.

10 Limitations and Open Problems

10.1 Trusted PSP

The largest remaining trust assumption is the PSP.

The construction proves that a registered PSP authenticated the accepted statement. It does not prove that the PSP reported truthfully.

The strongest future direction is therefore to replace or weaken this assumption through one of several approaches: a central-bank or rail-level signature; signed rail artifacts verified retrospectively; dual attestation from both sides of the payment; zkTLS/DECO [8] over an authoritative PSP endpoint; or a periodically signed accumulator of settled payment identifiers.

Of these, retrospective rail anchoring is particularly attractive because it does not have to remain on the critical path of settlement.

A signed membership root could allow an institution to prove, after the fact, that its settlement identifier belongs to the rail’s settled set while still exposing only the keyed tag on-chain.

Such a mechanism, however, requires an explicit completeness guarantee for the published root. Without completeness, non-membership in a published root does not prove non-settlement.

10.2 Instance-scoped uniqueness

The current spent-tag registry is scoped to the deployed (oracle, `Settle`) instance.

A production domain spanning multiple settlement contracts would benefit from a shared registry indexed by K_D .

This would elevate the uniqueness guarantee from instance scope to domain scope.

10.3 Domain-key compromise and rotation

The domain key k_D is a security boundary.

A compromised key can reveal the correspondence between observed `e2eld` values and historical settlement tags. It does not, by itself, forge PSP attestations or Groth16 proofs.

Key rotation is also non-trivial. Rotating k_D produces a new tag for the same `e2eld` and therefore cannot preserve the current spent-tag uniqueness property across domains automatically.

A production system must consequently treat key rotation as a domain transition rather than a transparent key-management operation.

10.4 Trusted setup

The reference settlement circuit uses a Groth16 setup with a single contributor followed by a beacon.

This is sufficient for the experimental artifact but is not a production-grade ceremony.

A production deployment should use a multi-party ceremony with independent contributors and a verifiable public randomness beacon.

The trusted setup limitation affects the soundness guarantees relying on Groth16; it does not affect the independent commitment-binding or signature-unforgeability properties.

10.5 Economic finality

The protocol’s `Irrevocable` state is not equivalent to economic finality on the underlying payment rail.

The reference deployment uses a 72-hour dispute window, while certain rail-level return and fraud mechanisms can remain effective for substantially longer periods.

A deployment requiring stronger asset protection must either extend its risk window or introduce a rail-level anchoring mechanism that can represent later reversals.

10.6 Atomic delivery-versus-payment

The two-phase construction provides an escrow commitment before settlement, which is the necessary ingredient for DvP composition.

However, a complete atomic composition with a confidential asset system has not yet been implemented and measured.

An integration with a system such as Zeto or Paladin would therefore be a meaningful next experiment rather than a result claimed by this paper.

10.7 Benchmark methodology

This paper reports on-chain gas costs but does not report off-chain proving and verification time. An earlier internal measurement pass was excluded because it consisted of

single warm runs rather than a statistically meaningful benchmark.

A proper benchmark should report proving and verification time over $N \geq 30$ warm runs per circuit, with median and dispersion, on fixed and disclosed hardware. This is left for future work.

Likewise, verifier comparisons across circuits should use the same generated verifier template before attributing gas differences to the number of public signals.

10.8 Well-determinedness analysis

The Ecne analysis provides evidence against one class of under-constrained circuits but does not prove functional correctness.

Future work should cross-check the result with an independent tool such as QED² [12], identify the currently unresolved signals in the KYC circuit, and extend the analysis to the optimized circuit representation.

10.9 Post-quantum audit disclosure

The ML-KEM [9] audit channel remains preliminary.

The full in-circuit ML-KEM binding compiles at approximately 1.62 million constraints, but end-to-end proving and on-chain verification measurements have not yet been established.

It should therefore remain outside the core security claim until those measurements and artifacts are complete. ML-KEM is not part of the security argument for cryptographic settlement binding; it is explored here solely as a preliminary post-quantum audit-disclosure extension.

11 Conclusion

We introduced a cryptographic settlement-binding construction for confidential EVM state.

The central problem is not to prove that a payment identifier exists, nor to replace an institutional settlement observer with zero knowledge. The problem is to establish a cryptographic correspondence between two events that occur at different times: a confidential obligation committed on-chain before external settlement; and an external settlement event observed and authenticated by an authorized institution.

The construction addresses this problem through a two-phase commitment.

Before settlement,

$$c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid})$$

provides an on-chain commitment to the agreed terms.

After settlement,

$$c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid})$$

incorporates the settlement-specific identifier.

A Groth16 relation proves that both commitments share the same confidential amount and charge identifier. A domain-keyed tag $\nu = \text{Poseidon}(\text{e2eld}, k_D)$ provides deterministic, instance-scoped single-use identification without exposing the underlying payment identifier. A registered PSP then authenticates the tuple $(c_0, \nu, pc, \text{true}, \text{deadline})$ through an EIP-712 receipt.

The resulting security statement is deliberately precise:

$$\text{accepted settlement} \Rightarrow \text{cryptographically consistent} \\ \text{confidential state} + \text{authenticated PSP statement.}$$

The construction does **not** claim to establish central-bank or payment-rail truth without trusting the attesting institution. Removing or weakening this assumption remains the principal open problem.

The Pix instantiation demonstrates that the construction can be implemented on a public EVM. The primary settlement circuit contains 1,251 R1CS constraints at -02, while the reference **finalize** transaction costs 353,062 gas on the XDC Apothem testnet. The implementation also demonstrates negative acceptance cases and provides circuit well-determinedness evidence under the stated methodology.

More broadly, the construction is intended as an interface between two otherwise separate systems:

$$\text{real-world settlement} \rightarrow \text{settlement binding} \rightarrow \text{confidential asset state.}$$

This interface can be instantiated for Pix and potentially for other instant-payment rails, and can serve as an upstream component for confidential asset systems.

The principal result is therefore not a new commitment primitive or a new zero-knowledge system. It is a formally characterized method for cryptographically binding an externally observed settlement event to a confidential on-chain state transition while revealing only application-required information.

A Pix Canonicalization Profile

This appendix specifies the concrete canonicalization profile used by the Pix instantiation of Section 6. It is a deployment profile and is not part of the cryptographic settlement-binding construction itself.

A.1 Counterparty Identifier

For CPF, the canonical representation is the 11 decimal digits of the identifier, with punctuation removed and no padding.

For CNPJ, the canonical representation is the 14-character identifier, upper-cased and with punctuation removed.

The resulting representation is encoded as the exact byte sequence consumed by the commitment function.

A.2 e2eld and txid

For the Pix deployment, `e2eld` and `txid` are represented by the exact bytes returned by the PSP API.

No case folding, trimming, or re-serialization is performed.

This rule avoids ambiguity in the byte representation used by the commitment function.

A.3 Provenance

The `e2eld` is generated by the payer’s PSP when the payment is initiated. It can therefore exist for a transaction that is later rejected as well as for one that settles.

Consequently, the presence of an `e2eld` is not treated as evidence of settlement. The settlement status is supplied through the authorized PSP attestation described in Sections 2–4.

A.4 Scope

Nothing in the core construction depends on these Pix-specific representations. Another instant-payment rail can replace the canonicalization profile with its own rail-defined identifier representation while preserving the settlement-binding construction.

B Threat Model Matrix

B.1 Trust Decomposition

The construction intentionally decomposes the final claim into independently attributable components.

This decomposition is the security boundary used by the formal analysis in Section 4.

C Ecne Constraint-Analysis Methodology

This appendix documents the methodology used to analyze the zero-knowledge circuits with Ecne. The purpose is to make the reported results reproducible and to prevent an overly strong interpretation of the tool’s verdict.

C.1 Scope of the Analysis

Ecne is used here to analyze whether the generated R1CS is well-determined, i.e., whether the signals relevant to the declared targets are uniquely constrained by the system.

This is an under-constraint analysis.

It is **not** a proof of:

- functional correctness of the circuit;
- correctness of the protocol specification;
- cryptographic security of Poseidon or Groth16;

- correctness of the Solidity verifier;
- truthfulness of the PSP attestation; or
- security of the complete application.

Accordingly, this paper refers to the result as **constraint analysis**, rather than formal verification.

This analysis is also distinct from static-analysis linting of the Circom source itself, such as Circomspect [13], which flags coding-pattern risks (for example, unconstrained signals introduced by common template misuse) without solving the resulting R1CS. The two approaches are complementary: a circuit can pass a linter and still be under-constrained, or fail a linter check while remaining fully constrained.

C.2 Avoiding the Vacuous-Target Problem

A direct Ecne run over a circuit written with public inputs constrained through equality statements can produce a misleadingly positive result.

In the original circuit structure, a public value may be supplied as an input and constrained with a relation such as

$$c === H(w).$$

Depending on compilation, the resulting R1CS can expose no public outputs to the Ecne target set. If the target set is empty, the criterion

$$targets\ solved = targets$$

reduces to $0 = 0$. The resulting “sound” verdict is therefore vacuous.

We therefore do not report the raw verdict alone.

C.3 Corrected Analysis

For the analysis reported in the paper, dedicated `_out` variants of the circuits were used.

The derived public signals were declared as explicit outputs, while preserving the constraint semantics of the original circuits. The analysis was performed on the unoptimized `-O0` R1CS because the optimizer can fold linear structure that Ecne’s symbolic analysis relies upon.

The primary settlement circuit was analyzed as `settlement_verify_v2_out`, and the KYC inclusion circuit as `kyc_inclusion_verify_out`.

C.4 Results

The `O0` settlement result solves all five declared targets and all reported signals.

The KYC circuit has one declared target, which is solved. The remaining 18 signals reported by Ecne as undetermined are therefore not among the declared targets. Those signals have not yet been individually mapped against the generated symbol file.

Adversary / Failure	Protected Property	Prop- Mitigation	Residual Limitation
False proof	ZK consistency	Groth16 knowledge soundness	Depends on the trusted setup
Different amount in c_0 and c	Two-phase binding	Same private amount is constrained in both commitments	None under the stated circuit and hash assumptions
Different operation identifier in c_0 and c	Operation binding	Same private txid is constrained in both commitments	Assumes identifiers are not legitimately reused across charges
Reuse of a settlement tag	Single use	Write-once oracle attestation, with <code>tagUsed</code> as contract-level defense in depth	Scoped to the deployed (oracle, <code>Settle</code>) instance pair
Replay across domains	Domain separation	$K_D = \text{Poseidon}(k_D)$ and $\nu = \text{Poseidon}(e2eld, k_D)$	Global uniqueness across domains is not provided
Forged PSP receipt	Attestation authentication	EIP-712 + ECDSA recovery + registered-attestor check	Does not protect against a malicious registered PSP
Expired receipt <i>settled = false</i> receipt	Temporal validity Settlement acceptance	Receipt deadline checked on-chain Finalization requires <i>settled = true</i>	Deadline policy is deployment-specific Truthfulness remains an institutional assumption
Wrong c_0 in receipt	Value/operation consistency	Receipt c_0 must equal locked c_0	A malicious PSP can still sign an incorrect statement
Wrong counterparty commitment	Counterparty binding	Receipt pc must equal proof pc	Correctness depends on the PSP's attestation
Prover chooses a favorable counterparty	Predicate binding	pc is a public circuit signal and signed receipt field	Consumer must check the predicate against the settled operation
$e2eld$ alone presented as settlement evidence	External-event authentication	PSP attestation required	No direct rail proof exists in the construction
Chain observer learns amount	Confidentiality	Amount appears only inside ZK witness and commitments	Side channels and external information may still correlate operations
Chain observer learns $e2eld$	Confidentiality	$e2eld$ appears only inside c and ν	Domain-key holders can deliberately link tags to $e2eld$ values
Domain key compromise	Selective linkability	Key is never public	Historical tags in that domain may become linkable
Per-operation salt compromise	Commitment hiding	Fresh salts	Exposure affects the corresponding operation
PSP signing-key compromise	Attestation integrity	Key revocation and receipt deadlines	Attacker may forge attestations during the usable window
PSP refuses to attest	Liveness	Lock timeout and cancellation	Settlement cannot be finalized without attestation
Front-running <code>finalize</code>	Operation ownership	<code>finalize</code> changes the already-selected operation and does not assign benefit to <code>msg.sender</code>	Downstream applications must preserve this invariant
External refund after finalization	Economic finality	Configurable reversal/dispute window	Rail-level return mechanisms may outlive the protocol window
Registry-owner compromise	Governance	Administrative controls	Production deployment should use stronger governance
Trusted-setup compromise	Proof soundness	Production ceremony required	Reference deployment uses a research-prototype ceremony
Central-bank/rail statement	External-world truth	Not cryptographically mitigated	Requires stronger rail-level evidence or additional attestors

Table 4: Threat model matrix.

The O2 settlement analysis is reported as **inconclusive**, not unsound. Ecne solved four of the five targets but could not determine the remaining target under the optimized constraint system.

C.5 Interpretation

The reported results provide evidence against one important class of implementation error: under-constrained R1CS signals.

They do not establish that the circuit computes the intended predicate.

For example, a circuit can be fully determined while computing an incorrect hash relation. Functional correctness therefore remains established by the circuit specification, manual audit, implementation cross-checks, and the cryptographic arguments of Section 4 rather than by Ecne alone.

A different under-constraint analysis, such as the techniques represented by QED² [12], would provide an independent cross-check. Such an analysis has not been

included in the present evaluation.

C.6 Relationship to the Main Security Argument

The Ecne analysis is complementary to the formal security analysis.

Theorems in Section 4 rely on the specified cryptographic assumptions and on the deterministic correctness of the deployed contract and circuit.

Ecne does not replace those assumptions and is not included as a term in the reduction.

Its role is narrower: it provides additional implementation-level evidence that the R1CS does not contain an obvious under-constrained signal under the stated analysis methodology.

References

- [1] Jens Groth. *On the Size of Pairing-Based Non-Interactive Arguments*. In *Advances in Cryptology — EUROCRYPT*

Claim	Evidence
Same amount and operation identifier occur in both commitments	Zero-knowledge proof
Settlement tag is derived from the settlement identifier	Zero-knowledge proof
Settlement tag is single-use	Oracle write-once attestation + on-chain tagUsed defense in depth
e2eld is bound to the accepted confidential state	Zero-knowledge proof + PSP attestation
Counterparty commitment is the one accepted for settlement	PSP attestation + on-chain cross-check
External settlement actually occurred	PSP attestation under institutional assumption I_1
Central-bank / rail infrastructure itself attests the event	Not established by this construction
Operation cannot remain locked forever	Protocol timeout
Settlement is economically irreversible	Not established cryptographically

Table 5: Trust decomposition.

Circuit	Opt.	Declared	Solved	Verdict
settlement_verify_v2_out	O0	5	5	sound
kyc_inclusion_verify_out	O0	1	1	sound
settlement_verify_v2_out	O2	5	4	inconclusive

Table 6: Ecne results by circuit and optimization level. Signals: 4,229/4,229 (O0 settlement), 18,230/18,248 (O0 KYC), 1,016/1,260 (O2 settlement).

- 2016, LNCS 9666, pp. 305–326. Springer, 2016. IACR ePrint 2016/260.
- [2] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schafner. *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. In *USENIX Security Symposium*, 2021.
- [3] M. Yaksetig and J. Xu. *Rayls: A Novel Design for CBDCs*. IACR ePrint 2025/1639, 2025.
- [4] M. Yaksetig, P. M. F. Pereira, S. Yang, M. Nejadgholi, and J. Xu. *Rayls II: Fast, Private, and Compliant CBDCs*. IACR ePrint 2025/1638, 2025.
- [5] LF Decentralized Trust, Paladin Project. *Zeto: Privacy-preserving implementations of fungible and non-fungible tokens, using UTXO as the underlying transaction model*. GitHub repository, accessed September 2026. <https://github.com/LF-Decentralized-Trust-labs/paladin>.
- [6] ZKP2P / Peer. *Peer Documentation*. Documentation portal, accessed September 2026.
- [7] Digital Asset. *Canton Network: A Network of Networks for Smart Contract Applications*. White paper, updated January 2024.
- [8] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, and A. Juels. *DECO: Liberating Web Data Using Decentralized Oracles for TLS*. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2020.
- [9] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. August 2024. <https://csrc.nist.gov/pubs/fips/203/final>.
- [10] A. R. Choudhuri, S. Garg, M. Gregoire, K. Lee, M. Lodder, H. Montgomery, G. V. Policharla, and J. Zhang. *Qurrency: A Quantum-Secure, Private, and Auditable Platform for Digital Assets*. IACR ePrint 2026/015, 2026.
- [11] F. Wang. *Ecne: An Engine for Verifying the Soundness of R1CS Constraints*. 0xPARC, 2022. <https://github.com/franklynwang/EcneProject>.
- [12] S. Pailoor, Y. Chen, F. Wang, C. Rodríguez, J. Van Gaffan, J. Morton, M. Chu, B. Gu, Y. Feng, and I. Dillig. *Automated Detection of Underconstrained Circuits for Zero-Knowledge Proofs*. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2023. IACR ePrint 2023/512.
- [13] Trail of Bits. *Circomspect: A Static Analyzer and Linter for the Circom Zero-Knowledge DSL*. 2022. <https://github.com/trailofbits/circomspect>.
- [14] Banco Central do Brasil. *Catálogo de Serviços do SFN, Volume VI — Pagamentos Instantâneos*, versão 5.12.
- [15] Receita Federal do Brasil. *Instrução Normativa RFB nº 2.229, de 15 de outubro de 2024*.